FACULTY OF ECONOMICS AND BUSINESS ADMINISTRATION

SIMULATION MODELLING AND ANALYSIS

TRAJECTORIES OF MARKOV CHAINS – EXERCISE 10

Group 8

Cédric Desrumaux
Eva Delorge
Febe De Smet
Hanne De Wilde
Louise Van den Braembussche
Senne Evenepoel

Prof. Broos Maenhout
Academic Year 2024-2025

GHENT UNIVERSITY

1 INTRODUCTION

This report presents a car rental system with multiple locations that can be modelled using a Markov Chain, as the future state of the system depends only on the current distribution of cars and not on past states. We constructed a state transition diagram and a transition matrix to visualize the system. To analyse the system performance, we applied three estimation methods: the Replication Method, the Batch Means Method, and the Regeneration Method.

2 TRANSITION DIAGRAM AND TRANSITION MATRIX

To visualize the car rental dynamics, we created a transition diagram (*Figure 1*) and a one-step transition matrix (*Figure 2*) that represent the movement of cars between the four cities: Phoenix, Denver, Chicago, and Atlanta. Each node in the diagram represents a city, while directed edges indicate possible car transitions based on given probabilities. In the transition matrix, each row represents a city's current state, and each column represents the probability of transitioning to another city or staying in the same location.

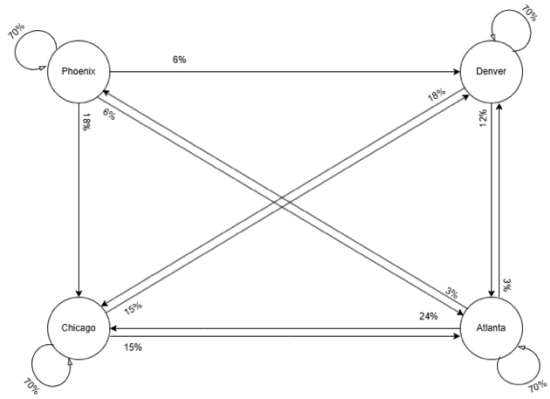


Figure 1. Transition Diagram.

$$P = \begin{bmatrix} 0.7, & 0.06, & 0.18, & 0.06 \\ 0.0, & 0.7, & 0.18, & 0.12 \\ 0.0, & 0.15, & 0.7, & 0.15 \\ 0.03, & 0.03, & 0.24, & 0.7 \end{bmatrix}$$

Figure 2. Transition Matrix.

3 METHODOLOGY

The system is modeled as a finite-state discrete-time Markov Chain with four states representing rental offices. The transition probabilities define car movements. To obtain reliable estimates, we implemented sufficient simulation steps to allow stabilization, a warm-up period to remove transient effects, and appropriate estimators to analyze dependent observations.

We performed three different methods to estimate the steady-state distribution of rental cars in the cities. The full code for each method can be found in Appendices 1, 2, and 3, respectively.
For the Replication Method, we ran 10 independent simulations, each lasting 1000 weeks, with cars randomly assigned to locations based on an equal probability distribution. At each step, car movements were determined by sampling from the transition matrix, and the results were averaged across all simulations. The running averages for this method, along with the confidence interval, can be found in Appendix 4.
For the Batch Means Method, we conducted a single long-run simulation of 100,000 weeks, divided into batches of 1000 weeks. The average car distribution within each batch was recorded to reduce correlation effects between observations, and the final estimate was obtained by averaging across all batches.

In the Regeneration Method, we selected Phoenix as the regeneration state due to its stable probability close to the initial value. A new cycle was recorded whenever the car distribution in Phoenix returned within a small tolerance of its initial probability, and the time spent in each state was tracked across all cycles to compute the steady-state distribution.

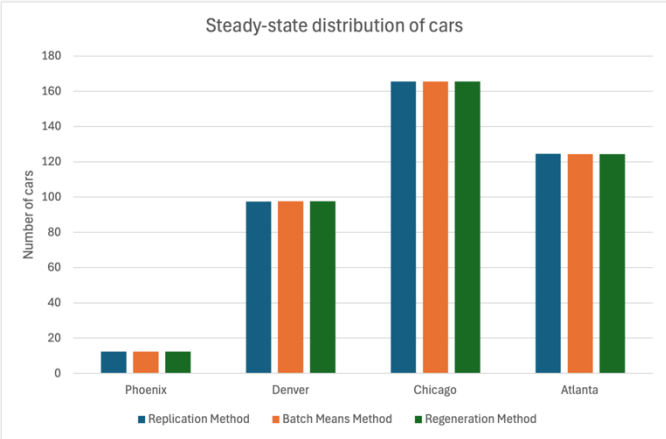
RESULTS AND ANALYSIS

3.1 Transient Period Analysis

To determine the appropriate transient period and avoid bias caused by the initial car distribution, we analysed the system's convergence towards the steady state in two ways.
First, we visually inspected the moving averages of the car distribution over the 1000-week simulation. This analysis indicated that the system reaches steady state after approximately 200 weeks.
Secondly, we applied the Algorithm of Welch, a more formal method, to detect the end of the transient period. We chose a window size w = 10 and computed, based on this window size, the moving average $\overline{Y}_t(w)$ as a time series. The results showed that the system reaches the steady state after approximately 30 weeks.
To ensure robustness and eliminate any residual bias, we decided to proceed our analysis with the visually determined transient period (200 weeks) to guarantee that all steady-state estimates are based on data collected after the transient effects have been cleared.

3.2 Steady-state Distribution

Graph 1 shows the steady-state distribution of rental cars in the four cities, based on three different methods: Replication, Batch Means, and Regeneration. As observed, the results from all three methods are very close to each other (Phoenix: 12, Denver: 98, Chicago: 166, Atlanta: 124), with only minor variations. This consistency indicates that the system has converged to a stable steady state, and the choice of method does not significantly affect the outcome. The closeness of the results suggests that the model is robust, and the steady-state estimates can be considered reliable and unbiased, providing a strong foundation for any further analysis or decision-making.



Graph 1. Steady-state distribution.

3.3 Impact of Initial Conditions

When starting the simulation with an initial state where each of the four cities (Phoenix, Denver, Chicago, Atlanta) has 100 cars, resulting in a uniform distribution, it is evident that this initial condition impacts the simulation model. During the first weeks, referred to as the transient period, the moving averages are far from the true steady-state distribution. To ensure an unbiased and accurate steady-state estimation, we decided to discard the first 200 weeks based on our graphical inspection. Although using Welsh's algorithm, one could consider removing just 30 weeks, the initial conditions only affect the first 200 weeks of the simulation. This initial bias influences the moving averages and, consequently, the steady-state distribution, making it unreliable if included in the analysis.

APPENDIX

1. Replication Method

```python
import numpy as np
import pandas as pd
import os

def initialize_simulation(M=4, N=400, T=1000):
    P = [0.25, 0.25, 0.25, 0.25]

    r = np.array([
        [0.7, 0.06, 0.18, 0.06],
        [0.0, 0.7, 0.18, 0.12],
        [0.0, 0.15, 0.7, 0.15],
        [0.03, 0.03, 0.24, 0.7]
    ])

    n = np.zeros((M, T+1), dtype=int)
    initial_distribution = np.random.choice(M, N, p=P)
    for location in initial_distribution:
        n[location, 0] += 1

    return M, N, T, r, n

def run_simulation(M, N, T, r, n):
    for t in range(T):
        new_n = np.zeros(M, dtype=int)
        for i in range(M):
            for _ in range(n[i, t]):
                next_station = np.random.choice(M, p=r[i])
                new_n[next_station] += 1

        n[:, t+1] = new_n
    return n

def save_results(all_results, file_path):

    final_df = pd.concat(all_results, ignore_index=True)


    final_df.to_csv(file_path, index=False)
    print(f"Results saved to {file_path}")
    simulations = 10
    all_results = []

    for sim_id in range(1, simulations + 1):
        M, N, T, r, n = initialize_simulation()
        n = run_simulation(M, N, T, r, n)


        weeks = np.arange(1, T + 1)
        df = pd.DataFrame({
            "Simulation": sim_id,
            "Week": weeks,
            "Phoenix": n[0, 1:],
            "Denver": n[1, 1:],
            "Chicago": n[2, 1:],
            "Atlanta": n[3, 1:],
        })


        all_results.append(df)


    file_path = "/Users/febedesmet/Documents/School/1e MASTER HIR-ITM/SEMESTER 2/Simulation Modelling and Analysis/Exercises/Exercise 2/MC_results.csv"
    save_results(all_results, file_path)

if __name__ == "__main__":
    main()
```

2. Batch Means Method

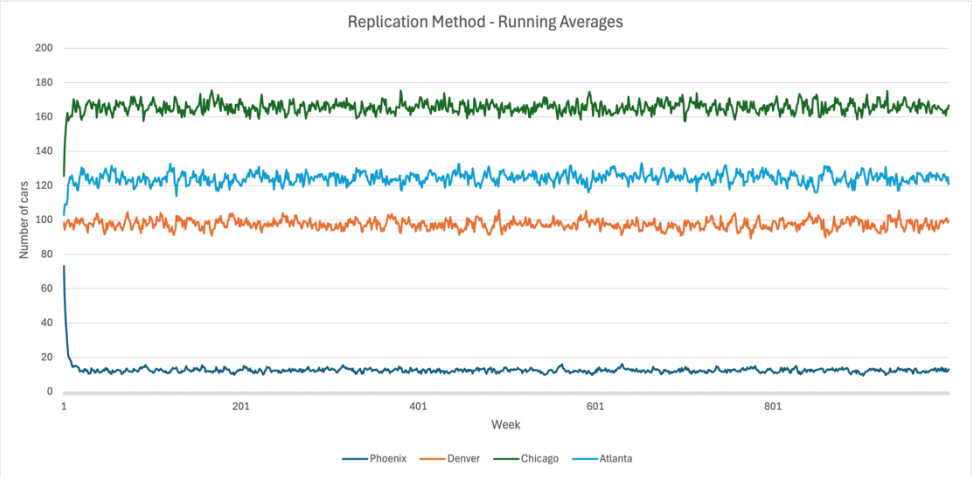
```python
# Re-import necessary libraries after execution state reset
import numpy as np

# Define the transition matrix
r_new = np.array([
    [0.7, 0.06, 0.18, 0.06],  # Phoenix
    [0.0, 0.7, 0.18, 0.12],  # Denver
    [0.0, 0.15, 0.7, 0.15],  # Chicago
    [0.03, 0.03, 0.24, 0.7]   # Atlanta
])

# Parameters for batch mean method
num_weeks = 100000  # Total number of weeks to simulate
batch_size = 1000  # Number of weeks per batch
num_batches = num_weeks // batch_size  # Total number of batches

# Initialize car counts (start with equal distribution)
car_counts = np.array([0.25, 0.25, 0.25, 0.25])  # Probability distribution

# Store batch averages
batch_averages = np.zeros((num_batches, 4))

# Run the simulation
for batch in range(num_batches):
    batch_sum = np.zeros(4)  # Sum of car distributions within the batch

    for _ in range(batch_size):
        car_counts = np.dot(car_counts, r_new)  # Apply transition matrix
        batch_sum += car_counts

    # Store the average over the batch
    batch_averages[batch] = batch_sum / batch_size

# Compute the final steady-state estimate as the average over all batches
steady_state_batch_mean = np.mean(batch_averages, axis=0)
steady_state_batch_mean
```

```python
# Run the simulation
for batch in range(num_batches):
    batch_sum = np.zeros(4)  # Sum of car distributions within the batch

    for _ in range(batch_size):
        car_counts = np.dot(car_counts, r_new)  # Apply transition matrix
        batch_sum += car_counts

    # Store the average over the batch
    batch_averages[batch] = batch_sum / batch_size

# Compute the final steady-state estimate as the average over all batches
steady_state_batch_mean = np.mean(batch_averages, axis=0)

# Print results
locations = ["Phoenix", "Denver", "Chicago", "Atlanta"]
print("Steady-State Distribution using Batch Mean Method:")
for loc, prob in zip(locations, steady_state_batch_mean):
    print(f"{loc}: {prob:.4%}")  # Print as percentage with 4 decimal places
```

3. Regeneration Method

```python
# Re-import necessary libraries after execution state reset
import numpy as np

# Define the transition matrix
r_new = np.array([
    [0.7, 0.06, 0.18, 0.06],  # Phoenix
    [0.0, 0.7, 0.18, 0.12],  # Denver
    [0.0, 0.15, 0.7, 0.15],  # Chicago
    [0.03, 0.03, 0.24, 0.7]   # Atlanta
])

locations = ["Phoenix", "Denver", "Chicago", "Atlanta"]
# Choose a regeneration state (Phoenix in this case)
regeneration_state = 0

# Reinitialize variables
num_weeks = 100000  # Total number of weeks to simulate
car_counts = np.array([0.25, 0.25, 0.25, 0.25])  # Start with uniform distribution
total_time_in_states = np.zeros(4)  # Track total time spent in each state
num_cycles = 0  # Count regeneration cycles

# Run the simulation
cycle_time = 0  # Time since last regeneration
for _ in range(num_weeks):
    car_counts = np.dot(car_counts, r_new)  # Apply transition matrix
    total_time_in_states += car_counts  # Accumulate time spent in each state
    cycle_time += 1

    # Detect regeneration when Phoenix's probability is close to its initial value
    if np.isclose(car_counts[regeneration_state], 0.25, atol=0.01):
        num_cycles += 1
        cycle_time = 0  # Reset cycle time

# Compute steady-state estimates using regeneration method
steady_state_regeneration = total_time_in_states / num_weeks

# Print results
print("Steady-State Distribution using Regeneration Method:")
for loc, prob in zip(locations, steady_state_regeneration):
    print(f"{loc}: {prob:.4%}")  # Print as percentage with 4 decimal places
```

4. Replication Method



Without transient period (N1=200)				
Average	**Phoenix**	**Denver**	**Chicago**	**Atlanta**
Run 1	12,65	97,54	165,55	124,26
Run 2	12,23	97,31	166,06	124,40
Run 3	11,93	97,32	164,87	125,88
Run 4	12,49	96,57	166,03	124,91
Run 5	12,05	97,90	166,21	123,84
Run 6	12,30	97,54	165,26	124,90
Run 7	12,52	96,87	166,27	124,34
Run 8	12,31	97,26	164,94	125,49
Run 9	12,68	98,60	165,48	123,24
Run 10	12,45	97,03	165,48	125,03
Avg	12,36	97,40	165,62	124,63
St dev	0,25	0,56	0,51	0,78
CI-	12,16	96,94	165,20	124,00
CI+	12,56	97,85	166,03	125,26
in %	3,0900	24,3488	41,4040	31,1572